

Business Report 2025



SHOPSTER

BUY ANYTIME ANYWHERE

APRIL 20

Shopster LLC

Authored by: Vinay R. Pai

To: Emily Watson (IT Technical Lead – UK)



Index

Introduction3

Technical Specification.....4

Systems and Subsystems5

Data Flow and Information exchange6

Data Type, Frequency, Speed and Volume Details7

Failure points and Mitigation Strategy9

Load Management.....11

Cost Implication and Optimization.....12

Unified Payment Solution14

Conclusion15

Introduction

We at Shopster are thrilled to announce the launch of our new e-commerce platform, designed to cater to customers all around the globe and to support high-traffic events like Black Friday. The new system is resilient, scalable, and provides seamless transactions.

Our new platform is designed with a user-centric approach, ensuring intuitive navigation and a seamless shopping experience. It employs responsive design principles, allowing it to adapt to various devices and screen sizes. Real-time feedback during transactions enhances user confidence, and personalized recommendations increase engagement.

Our Goal is to:

1. To keep 99.99% uptime – handle peak traffic loads seamlessly
2. Maintain payment success rate of over 98% to deliver smooth checkout experience. Ensure Payment gateways are supported by most of the banks in the country.
3. Ensure our user data is secure and meets legal compliance
4. Provide Good UI experience and Maintain core functionalities during failures
5. Scale to international regions in near future.

User Base:

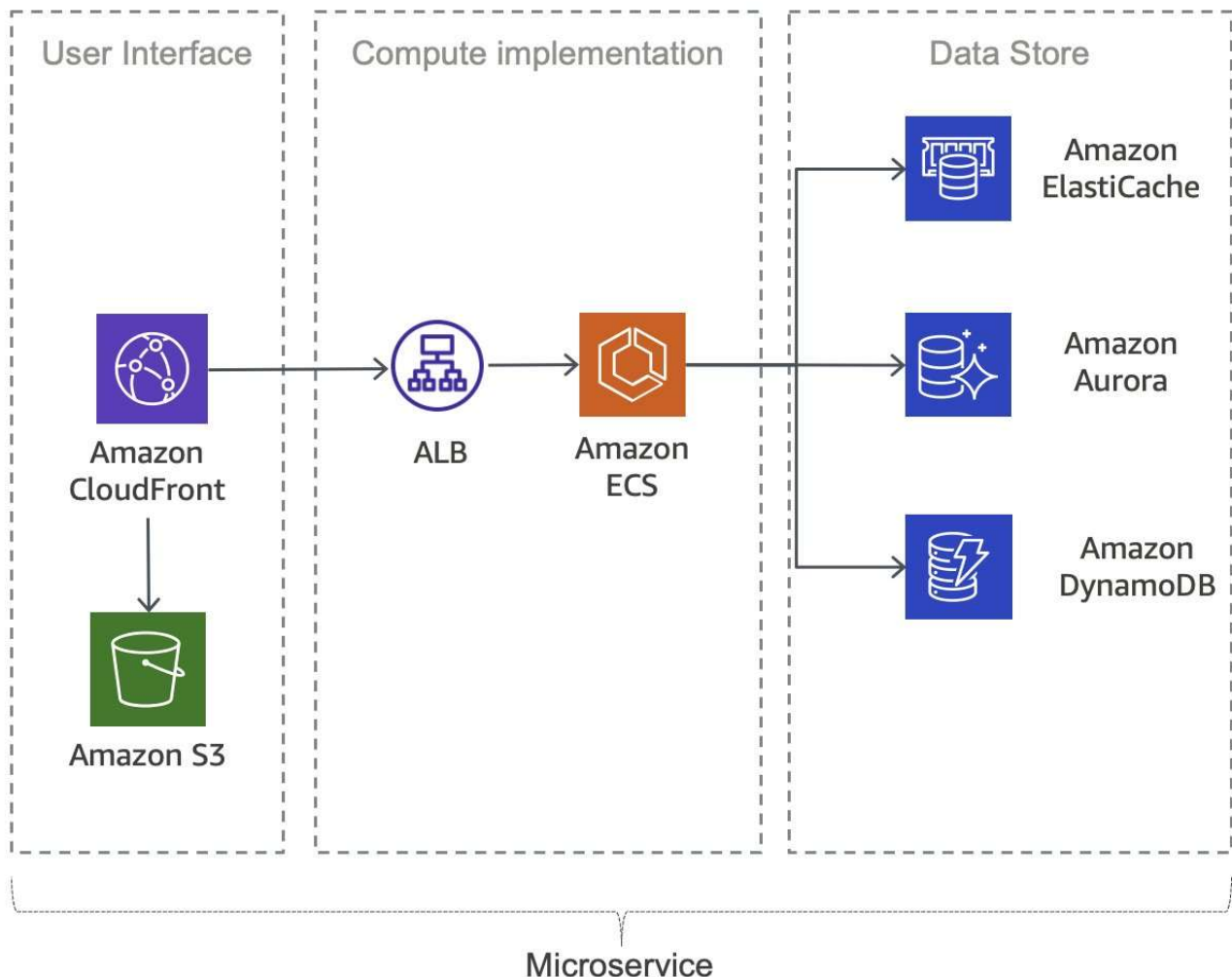
1. Expected Daily Active Users: 1 million
2. Peak Concurrent Users: 250,000
3. Transactions per Second (TPS): 10,000
4. Supported Platforms: Web, Mobile (iOS & Android)

Technical Specification

The platform is built using AWS microservices architecture, which allows for modular development and maintenance.

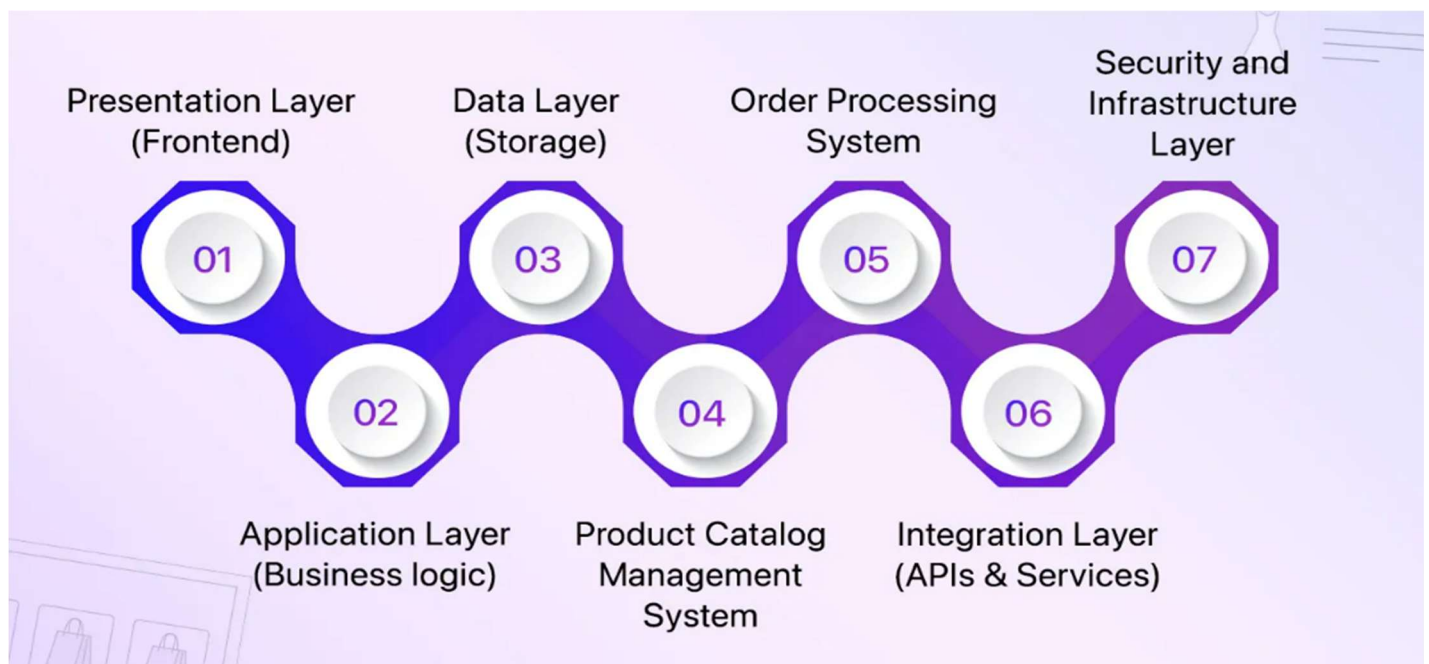
It utilizes AWS cloud infrastructure for scalability, with integrated CDN for content delivery optimization.

Technologies include container orchestration for resource management (AWS- EKS and ECS) and using Data Stores (ElastiCache, Aurora and Dynamo DB) for efficient data handling and predictable performance.



Systems and Subsystems

1. Presentation Layer - Frontend (Windows, Web, iOS, Android) – Includes UI, Product Catalog, Shopping Cart, User Authentication, Static pages.
2. Application Layer (Business Logic) – Includes Cart administration, order pipelines, user authentication protocols and business protocols.
3. Data Layer (Storage) – Includes Product catalog, customer profiles, inventory details and transaction records.
4. Product Catalog Management System – Includes production information, digital assets, categories and pricing models and inventory tracking.
5. Order Processing System – Includes Shopping cart activity and progressing to collecting and distribution as well as return management.
6. Integration Layer (APIs and Services) – Connects to external payment gateways, shipping providers, CRM platforms and marketing tools
7. Security and Infrastructure Layer – This enables encryption, authentication protocols and cloud infrastructure management.

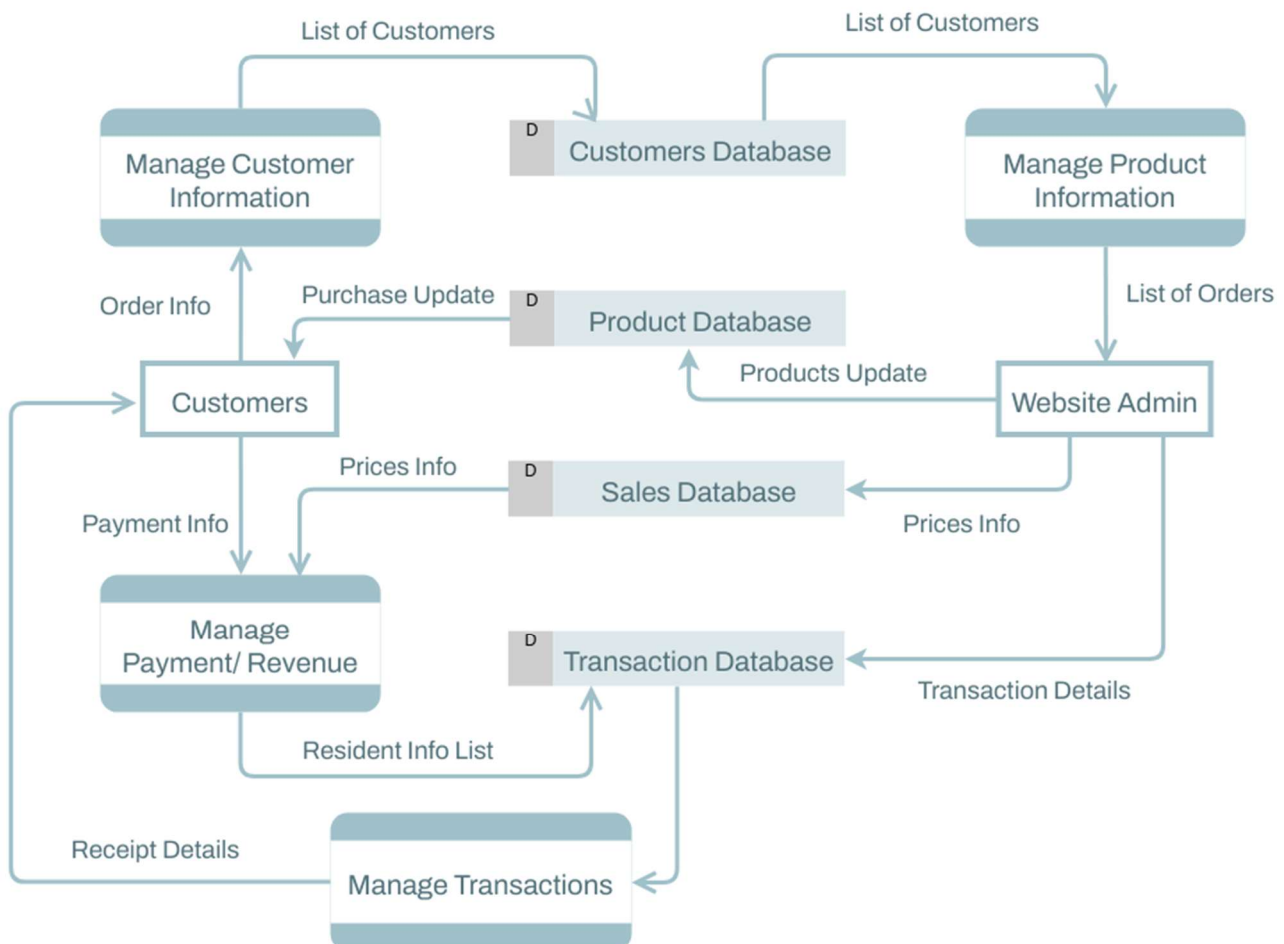


Data Flow and Information exchange

Types of Data flowing through the system and subsystems

1. User & Account Information - Personal Data, Authentication Data, Preferences
2. Product & Catalog Data
3. Cart and checkout information
4. Transaction & Payment Data
5. Order Management
6. Logistics & Fulfillment Data
7. Customer Behavior & Analytics
8. Marketing & Communication

Below is a DFD between Customer, Product, Sales and Transaction Databases



Data Type, Frequency, Speed and Volume Details

Data Type	Frequency	Speed (Latency Requirements)	Volume (Approx. Daily)
User Page Views	Very High – Thousands per second	Real-time (<100ms)	Millions to tens of millions
Product Searches	High – Especially during sales/promotions	Low latency (<200ms)	Hundreds of thousands to millions
Add to Cart Events	High – During peak shopping hours	Near real-time	Hundreds of thousands
Orders/Purchases	Moderate – Higher during flash sales	Real-time	Tens to hundreds of thousands
Payment Transactions	Tied to purchases – High importance	Real-time / Secure (<1s)	Tens of thousands
Inventory Updates	Frequent – Especially with multiple channels	Near real-time	Thousands to hundreds of thousands
Product Reviews/Ratings	Moderate – User-driven	Low urgency (seconds to minutes)	Thousands
Customer Support Tickets	Variable – Depends on issues & scale	Minutes to hours	Hundreds to thousands
Promotions & Price Updates	Low to Moderate – Scheduled or ad hoc	Low latency (minutes)	Thousands

Data Type	Frequency	Speed (Latency Requirements)	Volume (Approx. Daily)
Log & Audit Data	Very High – Continuous generation	No real-time need (batch acceptable)	Gigabytes to terabytes
Clickstream Data	Very High – Every user interaction tracked	Real-time (for personalization/analytics)	Terabytes
Email/SMS Notifications	Burst-based – On trigger events	Near real-time	Thousands to hundreds of thousands

Failure points and Mitigation Strategy

Our new platform has built in resiliency and uses various mitigation strategies to overcome potential failure points. Few of them are listed below:

Failure Point	Description	Mitigation Strategy
Website Downtime	Website becomes unavailable due to server issues, traffic spikes, etc.	Use load balancers, CDNs, and auto-scaling; ensure high-availability architecture.
Slow Page Load Time	Pages take too long to load, causing poor UX and cart abandonment.	Optimize images, use caching, minimize HTTP requests, and deploy CDN.
Payment Gateway Failures	Transactions fail due to third-party payment processor issues.	Integrate multiple payment gateways; implement graceful fallback and retry logic.
Security Breaches	Hacking, data theft, or fraudulent transactions.	Use SSL/TLS, secure coding practices, regular audits, and compliance with PCI-DSS.
Inventory Inaccuracy	Showing incorrect stock levels, leading to customer dissatisfaction.	Use real-time inventory management systems; sync inventory across all platforms.
Poor Mobile Optimization	Mobile users face navigation or display issues.	Use responsive design and conduct regular mobile UX testing.
Checkout Process Failures	Complex or buggy checkout flows causing drop-offs.	Simplify forms, offer guest checkout, ensure error handling and field validation.
Search Functionality Issues	Inaccurate or slow search results frustrate users.	Use intelligent search algorithms (e.g., Elasticsearch), auto-suggestions, filters.

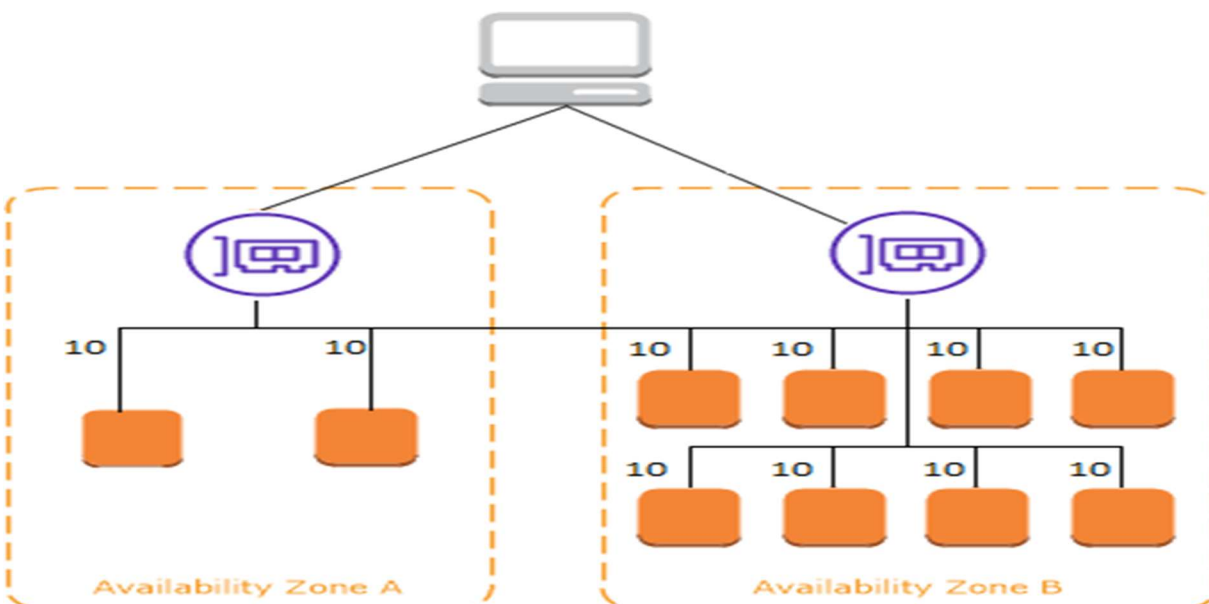
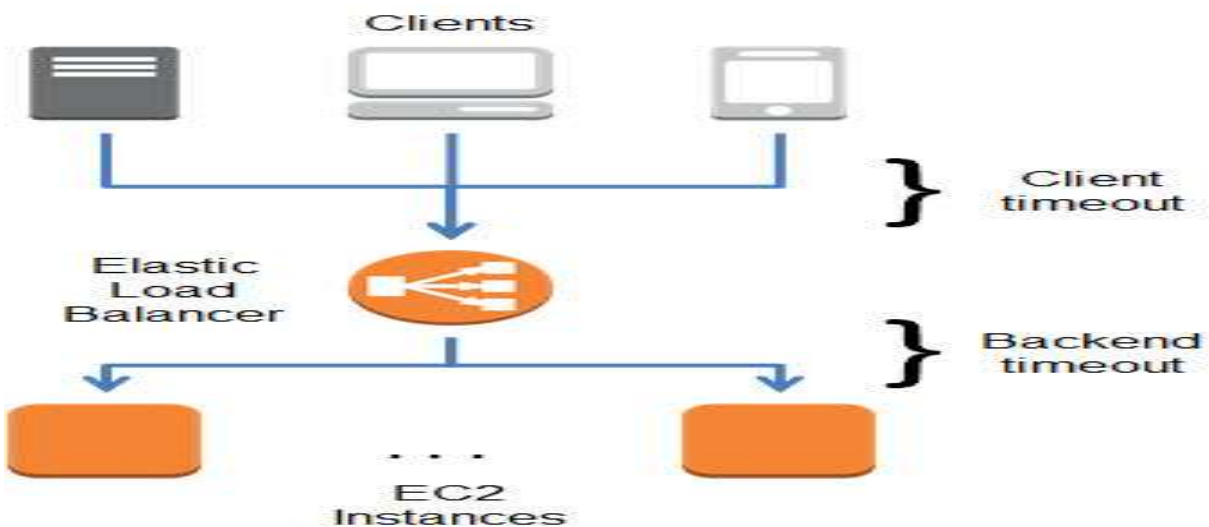
Failure Point	Description	Mitigation Strategy
Third-party Integration Failures	APIs for shipping, payment, or reviews break or respond slowly.	Monitor third-party APIs; implement retries and graceful degradation.
Cart Abandonment	Users add items to cart but don't complete purchase.	Use retargeting emails, exit-intent popups, persistent carts, and streamlined checkout.
Order Fulfilment Delays	Orders are not processed, shipped, or delivered on time.	Automate fulfilment, use reliable logistics partners, and track SLAs closely.
Customer Support Delays	Slow or poor support responses damage brand trust.	Use chatbots for FAQs, implement CRM systems, and train support staff efficiently.
Data Loss or Corruption	Critical data is lost due to errors or attacks.	Regular backups, data replication, and disaster recovery plans.
Scalability Limitations	System can't handle sudden spikes in traffic (e.g., sales events).	Use cloud-native, scalable architecture with performance testing before events.

Load Management

System behavior during average load and peak load

To handle varying levels of traffic, our platform incorporates a dynamic load balancing system, which intelligently distributes user requests across multiple servers.

This is done using Elastic Load Balancing (ELB), Amazon Auto Scaling and AWS Lambda Resilience is built in using Multi – AZ deployments



Cost Implication and Optimization

Cost optimization on our platform is all about balancing performance, scalability, and customer experience while minimizing unnecessary expenses. Here's a table breaking down our key cost areas and strategies we use to optimize them:

Cost Area	Description	Cost Optimization Strategies
Cloud Infrastructure	Servers, storage, bandwidth (e.g., AWS, Azure, GCP)	Use auto-scaling, reserved instances, spot instances, serverless architecture, and cost monitoring tools (e.g., AWS Cost Explorer).
CDN & Bandwidth	Content delivery and data transfer costs	Optimize images/files, enable compression, cache effectively, and use efficient CDNs (Cloudflare, Akamai).
Third-party APIs & Services	Payment gateways, search engines, recommendation engines	Evaluate ROI, consolidate vendors, use APIs with usage-based pricing, or build in-house if cheaper long-term.
Marketing & Ads	Paid ads, social media campaigns, affiliate programs	Use targeted marketing, A/B testing, retargeting strategies, and marketing automation tools.
Customer Support	Salaries, tools, and support channels	Implement AI chatbots, self-service help centres, and outsource where cost-effective.
Software Licenses	CMS, CRM, analytics, email marketing tools	Audit tools regularly, remove unused licenses, and switch to open-source or consolidated platforms.
Returns & Refunds	Cost of reverse logistics and lost sales	Improve product descriptions, sizing guides, and quality control to reduce returns.

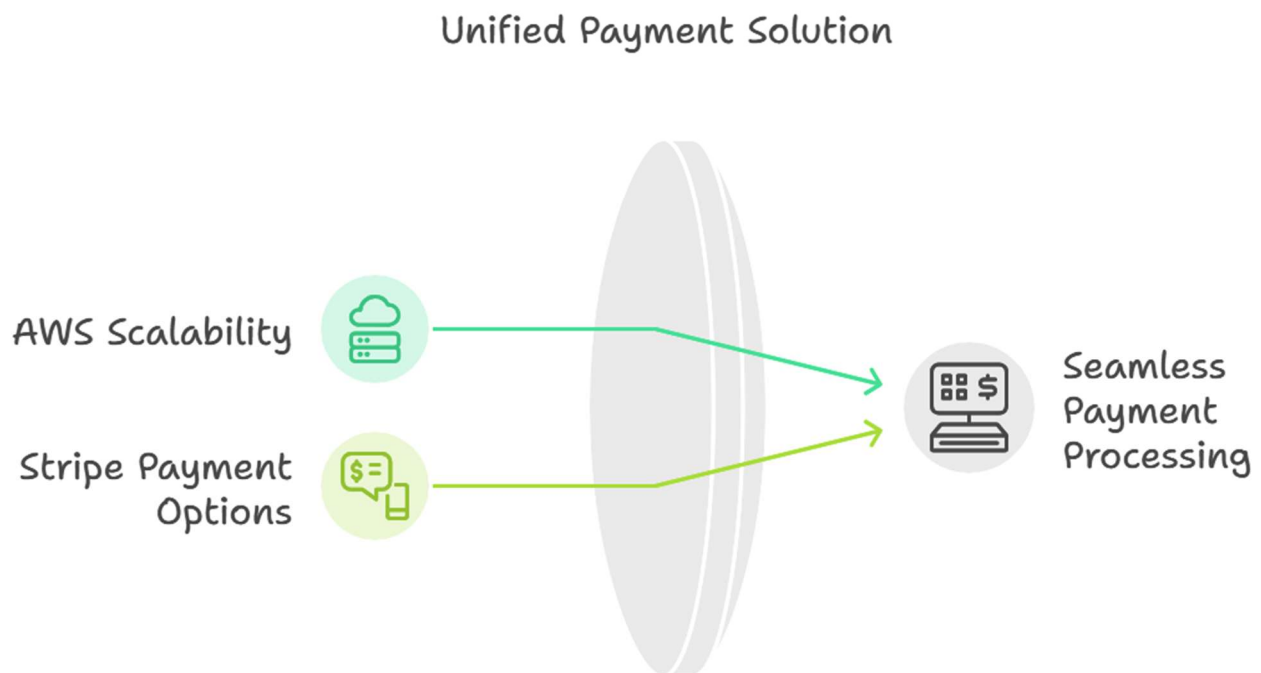
Cost Area	Description	Cost Optimization Strategies
Warehouse & Fulfilment	Storage, handling, packing, and delivery costs	Use Just-in-Time (JIT) inventory, optimize warehouse layout, and negotiate bulk shipping rates.
Development & Maintenance	Dev team salaries, testing, bug fixing, feature updates	Automate CI/CD pipelines, use agile methods, and prioritize features based on customer value.
Data Storage & Processing	Databases, analytics, backups	Archive cold data, use efficient storage tiers, compress logs, and offload to cost-effective cloud storage.
Fraud & Chargebacks	Costs from fraudulent transactions	Use machine learning-based fraud detection tools and multi-factor authentication.
Energy & Hosting	Physical servers, cooling (for on-prem)	Move to cloud or hybrid, and use energy-efficient hardware.

Unified Payment Solution

Our platform uses a Unified Payment Solution using AWS by designing a scalable, secure, and highly available architecture that can process payments across multiple channels (web, mobile, APIs) while integrating with various third-party payment gateways such as Stripe payment options.

We have also used Key Security Best Practices in our payment solution such as:

- Tokenization of card data before it reaches our system (use PCI-DSS compliant gateways).
- Encryption at rest and in transit using AWS KMS and TLS.
- IAM policies with least privilege for Lambda, API Gateway, and database access.
- WAF (Web Application Firewall) to prevent injection attacks or DDoS on payment endpoints.



Conclusion

In summary, our new platform is strategically developed to support high-traffic events with resilience and scalability. Its advanced features and user-centric design will ensure seamless transactions and enhanced customer satisfaction, positioning it as a market leader.